

Overall Design of Program

The primary objective of this assignment is to implement a lossy image compressor in Rust, focusing on integer and logical operations to manipulate image data. The project encompasses several components, including bit packing, transformation functions, and the core image compression and decompression logic. Each component will be designed, implemented, and tested individually, adopting a stepwise refinement approach to ensure a robust and efficient solution.

Each Separate Component

1. Bitpack Crate

Purpose:

- The Bitpack Crate is crucial for packing and unpacking bits into words, which is fundamental for the compression and decompression processes.

Design:

- Implement functions to pack and unpack bits as per the provided stub.
- Ensure error handling for overflow and underflow conditions.

Testing Plan:

- Unit tests to validate the correctness of packing and unpacking operations.
- Edge case testing for boundary values.
- Performance testing to ensure optimal bit manipulation speed.

2. Transformation Functions

Purpose:

- Implementing Discrete Cosine Transform (DCT) and RGB to Y/PR/PB color space transformation functions to facilitate image compression.

Design:

- Efficient algorithms for DCT and color space transformations.
- Handling different image dimensions and color intensities.

Testing Plan:

- Validate correctness with known input-output pairs.
- Performance testing to measure the speed of transformations.
- Inverse testing to ensure reversibility where applicable.

3. RPEG Program

The rpeg implementation comprises several components working together to accomplish image compression and decompression. The primary components include the bitpack module, the codec module, and the supporting libraries like csc411_arith, csc411_image, and csc411_rpegio. Each component plays a distinct role, and they interact through well-defined interfaces to accomplish the overall objective of image compression and decompression.

a. Bitpack Module

Architecture:

- Types:
 - Bitpack: **A structure to handle the packing and unpacking of bits.**
- Functions:
 - pack: **Packs multiple smaller integers into a single word.**
 - unpack: **Unpacks multiple smaller integers from a single word.**

Testing Plan:

- **Test the pack and unpack functions with a variety of input sizes and values to ensure correctness.**

- Universal Laws:
 - The unpacked values, when repacked, should yield the original word.

b. Codec Module

Architecture:

- Functions:
 - compress: **Compresses an image using specified algorithms and writes to file.**
 - decompress: **Decompresses a compressed image file back into its original form.**

Testing Plan:

- **Test the compress and decompress functions using a variety of image sizes, types, and compression settings.**
- Universal Laws:
 - Decompressing a compressed image should yield an image that is visually similar to the original, though not necessarily identical due to lossy compression.

c. CSC411 Arith Module (External)

Architecture:

This module provides arithmetic utilities essential for compression algorithms.

- Functions:
 - **arith_encode**: Encodes data using arithmetic encoding.
 - **arith_decode**: Decodes data using arithmetic decoding.

Testing Plan:

- Utilize this module in encoding and decoding processes, and verify the accuracy of the operations.

d. CSC411 Image Module (External)

Architecture:

This module provides functionalities for handling image data.

- **Types:**
 - **RgbImage:** A structure representing an RGB image.
- **Functions:**
 - **read:** Reads image data from a file.
 - **write:** Writes image data to a file.

Testing Plan:

- **Test reading and writing functions with a variety of image sizes and formats to ensure correctness.**

e. **CSC411 Rpegio Module (External)**

Architecture:

Handles input and output operations for the rpeg program.

- **Functions:**
 - **Various IO functions necessary for reading from and writing to files.**

Testing Plan:

- **Test the IO functions with a variety of input and output scenarios to ensure correctness and robustness.**

f. **Main Module**

Architecture:

- **Types:**
 - **Args:** A structure to handle command-line arguments.
- **Functions:**
 - **main:** Entry point for the rpeg program, orchestrating the compression or decompression process based on user input.

Testing Plan:

- **Test the main function with a variety of command-line arguments to ensure correct program behavior.**

In the implementation of rpeg, the main module will parse the command-line arguments and invoke either the compress or decompress function of the codec module. The codec module, in turn, will leverage the bitpack module for bit packing operations, and external modules csc411_arith, csc411_image, and csc411_rpegio for arithmetic operations, image handling, and IO operations respectively. Each module and function will be tested in isolation to ensure correctness, followed by integrated testing to ensure the seamless interaction of these components.

Information Loss Questions

1. An image is compressed and then decompressed. Identify all the places where information could be lost.

When using lossful compression, there are a number of different stages in the process where information can be lost. The first way information could be lost is during the “Trim” stage we discussed in class. In this stage, if the width or height is an odd number, we will end up losing information when the last row/column is trimmed. Another stage where we are likely to lose data is when we average the $\overline{P_B}$ and $\overline{P_R}$ (chroma) elements of pixels in the 2x2 block. Since we are getting the average of the chroma data, we are likely to lose color information for individual pixels within the block. The next stage, where a discrete cosine transform is used to transform the 4 luminance values into cosine coefficients, is another place where we could lose information. In this stage, information can be lost when we approximate the coefficient values or when we disregard high-frequency components to reduce data size.

2. Then it's compressed and decompressed again. Could more information be lost? How?

When the image is compressed and decompressed again, there is a potential for further information loss. The errors or approximations made during the first cycle of compression and decompression could be compounded during the second cycle. For instance, any rounding or truncation errors from the first cycle could lead to further loss or distortion of information in the second cycle. Further, if quantization is applied again during the second cycle of compression, this could lead to further reduction in precision, and hence more information loss.

Repeated application of Discrete Cosine Transform (DCT) and inverse DCT could introduce more artifacts, further degrading the image quality. Finally, the loss of high-frequency details or color precision in the first cycle could lead to a degraded baseline for the second cycle, meaning the image going into the second cycle is already of lesser quality, and the process could degrade it further.

Overall Architecture of Compression Algorithm:

1. Read Image
 - a. Read pixel data from a ppm to 2D array; array2.
2. Trim Image
 - a. Trim the last rows/columns of an image's width/height so that the remaining dimensions are divisible by 2 (or even).
 - i. This step is modifying the number of rows/columns/both in array2
 - b. A new trimmed array may need to be created.
3. Change to floating point representation of pixels
 - a. Hint: (Think about the ppm format and its denominator)
 - b. (Red, Green, Blue) -> (f32, f32, f32)
4. Red/Blue/Green into Component Video
 - a. Transform each pixel from RGB color space into component video color space.
 - b. (f32, f32, f32) -> (Y / P_B / P_R)
 - i. Y = Brightness of Green in image
 - ii. P_B = Difference in Brightness between Blue and Green
 - iii. P_R = Difference in Brightness between Red and Green
5. Group Pixels
 - a. Group together 2x2 blocks of pixels into a 32-bit word.
 - b. For the P_B and P_R (chroma) elements of the pixels, take the average value of the four pixels in the block.
 - i. We'll call these average values $\overline{P_B}$ and $\overline{P_R}$.
 - c. Convert the $\overline{P_B}$ and $\overline{P_R}$ elements to 4-bit values using the function:
 - i. `csc411_arith::index_of_chroma(x: f32) -> usize`
 - ii. Takes: a chroma value between -0.5 and +0.5
 - iii. Returns: a 4-bit quantized representation of the chroma value.
 - d. Transform the four Y-values into cosine coefficients a, b, c, and d.
 - i. Using a discrete cosine transform (DCT)
 - ii. Four Y (luminance/luma) values (of the pixels)
 - e. Convert the b, c, and ~d to five-bit signed values.
 - i. Assuming that they lie between -0.3 and 0.3.
 - Although these values can actually range from -0.5 to +0.5, a value outside the range ± 0.3 is quite rare. We are willing to throw away information in the rare cases in order to get more precision for the common cases.
 - ii. Pack a, b, c, d, $\overline{P_B}$, and $\overline{P_R}$ into a 32-bit word as follows:

Value	Type	Width	LSB
a	Unsigned scaled integer	9 bits	23
b	Signed scaled integer	5 bits	18
c	Signed scaled integer	5 bits	13
d	Signed scaled integer	5 bits	8
$\text{index}(\overline{P_B})$	Unsigned index	4 bits	4
$\text{index}(\overline{P_R})$	Unsigned index	4 bits	0

The *index* operation is implemented by “`csc411_arith::index_of_chroma`,” it quantizes the chroma value and returns the index of the quantized value in an internal table. To pack the codeword, we will use the bitpack crate we will develop in Part B.

6. Write a compressed binary image to standard output.
 - a. The header of the compressed binary image should be written by:
 - i. `println!("Compressed image format 2\n{} {}", width, height);`
 - b. This header should be followed by a sequence of 32-bit code words.
 - i. One for each 2-by-2 block of pixels.
 - ii. The *width* and *height* variables describe the dimensions of the original (decompressed) image, *after* trimming off any odd column/row.
 - Each 32-bit code word should be written to disk in big-endian order;
 - i.e. with the most significant byte first.
 -
 - c. The Rust standard library functions **from_be_bytes** and **to_be_bytes** will be useful here: https://doc.rust-lang.org/std/primitive.u32.html#method.from_be_bytes
 - i. You can write a single byte using `print!()`
 - ii. Code words should be written in **row-major** order; i.e.
 1. First the code word for the 2-by-2 block containing pixel (0, 0),
 2. Then the block containing pixel (2, 0), and so on.
7. The decompressor will be the inverse of the compressor:
 - a. Read the header of the compressed file to determine the width and height.
 - b. Since Rust lacks an equivalent to C’s **fscanf**, we may use the **scan_fmt** crate.
 - i. The trick is to use exactly the same string as is printed in the header, as in this trivial example:


```
#[macro_use] use scan_fmt;

fn main() {
    let (w, h) = scan_fmt!("Compressed image format 2\n{} {} \n", u32, u32);
    println!("{}", w.unwrap(), h.unwrap());
}
```

You will also need to include it in your dependencies in `Cargo.toml`:

```
[dependencies]
scan_fmt = "^0"
```
 - c. Allocate a 2D array of pixels of the given width and height.
 - d. Use the new **csc411_image::write()** function to write out an image to stdout.